

Event object basics in JavaScript

In this lesson, we will learn the basics of working with the `Event` object. This object contains information about the event that occurred. For example, if an element was clicked, we can find out the coordinates of this click, whether the `Ctrl`, `Alt` or `Shift` key was pressed at the time of the click etc.

Let's see how to get the `Event` object. Let's have a button:

```
1 <button id="elem">text</button>
```

Let some function be executed by clicking on this button:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4
5 });
```

There is already a `Event` object inside the bound function - we just don't know how to retrieve it yet. It retrieves as follows: when declaring our function, we need to pass any variable into it as a parameter (as a rule, `event` - but the name can be anything) and *the browser will automatically put the `Event` object into this variable*:

```
1 elem.addEventListener('click', function(event) {
2   console.log(event); // shows the object with the event
3 });
```

№ 1

⊗jsPmOEInr

Perform the described actions yourself and output the object with the event to the console. Study the structure of this object.

Event coordinates in JavaScript

Learn the theory at the following links:

[clientX](#) [clientX](#) [clientY](#) [pageX](#) [pageY](#)

№ 1

⊗jsPmOEEC

Make it so that when the mouse moves across the page, the cursor coordinates are displayed, similar to how it's done in the following sample:

open in a new child window

event.clientX property

The `event.clientX` property contains the coordinates of the mouse cursor along the X axis. To determine the coordinates, there are also `event.clientY`, `event.pageX`, `event.pageY` properties. Let's see the difference between `clientX/clientY` and `pageX/pageY`.

How `clientX` and `clientY` work: if you have a 1000 by 1000 pixels window and the mouse is in the center, then `clientX` and `clientY` will both be equal to 500. If you then scroll the page horizontally or vertically without moving the cursor, the values `clientX` and `clientY` will not change because they are relative to the window, not the document.

How `pageX` and `pageY` work: if you have a 1000 by 1000 pixels window and the cursor is in the center, then `pageX` and `pageY` will be equal to 500. If you then scroll the page 250 pixels down, then `pageY` becomes 750. Thus `pageX` and `pageY` contain the coordinates of the event, taking scrolling into account.

Syntax

```
1 event.clientX;
```

Example

When moving the mouse on the page, we will display its coordinates relative to the browser window (`clientX` and `clientY`):

```
1 <div id="elem">0 : 0</div>

1 let elem = document.getElementById('elem');
2
3 document.addEventListener('mousemove', function(event) {
4   elem.innerHTML = event.clientX + ' : ' + event.clientY;
5 });
```

Result of code execution:

open in a new child window

Event type in the Event object in JavaScript

The `Event` object also contains the type of the event that occurred. By type is meant the name `click`, `mouseover`, and so on. The `type` property contains the event type:

```
1 elem.addEventListener('click', function(event) {
2   console.log(event.type); // shows 'click'
3 });
```

№ 1

©jsPmOET

In the following code, the same handler is attached to two events:

```
1 <button id="elem">text</button>

1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', func);
4 elem.addEventListener('dblclick', func);
5
6 function func() {
7
8 }
```

Add a code to the function `func` so that when you click on an element, this element is colored green, and when double-clicked - red.

Event element in an Event object in JavaScript

The `Event` object also allows you to get the element where the event occurred. This element is contained in the property `target`. Why is this necessary if this element is contained in `this`?

The fact is that in fact `this` always contains the element *to which the event was attached*, and the `target` property - the element that was actually clicked. This real element may or may not match `this`.

Let's look at an example. Let's say we have a `div` with a paragraph inside it:

```
1 <div id="elem">
2   <p>text</p>
3 </div>
```

Bind the event to the div, but click on the paragraph. Obviously, clicking on a paragraph is also a click on a div, since the paragraph is contained in our div.

In the described case, it turns out that the `target` property will contain the end tag in which the event happened - that is, a paragraph, not a div. Let's make sure of this:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function(event) {
4   console.log(event.target); // will output our paragraph
5   console.log(this);        // will output our div
6 });
```

№ 1

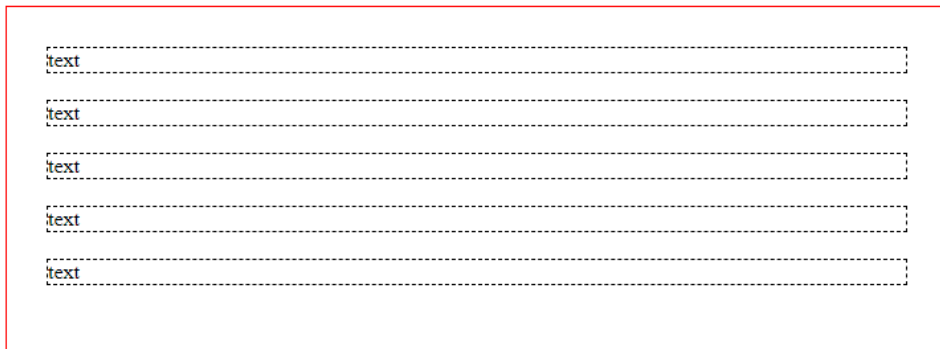
Let's say you have the list `ul` with `li` tags:

```
1 ul {
2   padding: 30px;
3   border: 1px solid red;
4 }
5 li {
6   list-style-type: none;
7   margin-bottom: 20px;
8   border: 1px dashed black;
9 }
```

```
1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Result of code execution:

Result of code execution:



Attach a click handler to the `ul` tag. In this handler, use the `tagName` property to check which tag was clicked. If the click was on the `li` tag, add an exclamation mark to the end of this tag text. And if the click was on the tag `ul` - show information about it in the console.

Getting pressed keys in JavaScript

Using the `Event` object, you can get the keys pressed. Let's see how it's done. Let's have an input:

```
1 | <input>
```

Get a reference to it into a variable:

```
1 | let elem = document.querySelector('input');
```

Let's hang an event on our input that fires on each keystroke:

```
1 | elem.addEventListener('keypress', function(event) {  
2 |  
3 | });
```

The `key` property of the object with the event will contain the pressed character:

```
1 | elem.addEventListener('keypress', function(event) {  
2 |   console.log(event.key);  
3 | });
```

And the `code` property will contain the code of the pressed key:

```
1 | elem.addEventListener('keypress', function(event) {  
2 |   console.log(event.code);  
3 | });
```

№ 1

⊗ jsPmOEPKG

Make an input that will display the values of the entered keys and their codes on input.

№ 2

⊗ jsPmOEPKG

Press various keys in the input and see what their values and codes are.

№ 3

⊗ jsPmOEPKG

Determine what code the `Enter` key will have.

№ 4

⊗ jsPmOEPKG

Determine what code the `BackSpace` key will have.

№ 5

⊗ jsPmOEPKG

Given a paragraph and an input. Text is entered into it and the key `Enter` is pressed. Make it so that at this moment the entered text falls into the paragraph under the input, and the contents of the input are cleared.

Modifier key tracking in JavaScript

Using the `Event` object, you can find out if the `Ctrl`, `Alt`, and `Shift` keys were pressed at the time of the event. This is done using the properties `ctrlKey`, `altKey` and `shiftKey` - they have the value `true` or `false` depending on whether this key was pressed at the time of the event or not.

Let's look at an example. Let's say we have the following button:

```
1 <button id="elem">text</button>
```

By clicking on the button, we will display a message about whether one of the keys `Ctrl`, `Alt` and `Shift` was pressed:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function(event) {
4   if (event.ctrlKey) {
5     console.log('Ctrl is pressed');
6   }
7
8   if (event.altKey) {
9     console.log('Alt is pressed');
10  }
11
12  if (event.shiftKey) {
13    console.log('Shift is pressed');
14  }
15 });
```

№1

⊗jsPmOEKM

Given an element. Make it turn red when clicked, but only if the **Alt** key is pressed at the time of the click.

№2

⊗jsPmOEKM

Let's say you have the list **ul** with tags **li**:

```
1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Make it so that when you click on any **li**, the number 1 is added to the end of its text if the **Ctrl** key is pressed, and the number 2 if **Shift** is pressed.

Canceling default action in JavaScript

Sometimes you need to use JavaScript to cancel the default action of a tag. For example, by clicking on a link, cancel the transition to this link. As a rule, this is required if we use this link just to run some code. In this case, the fact that when you click on the link will follow it, we do not need at all.

The default action is canceled using the **Event** object. To do this, it has a special method **preventDefault()**, which should be called anywhere in the event handler.

Let's try it in practice. Let's have a link like this:

```
1 <a href="/" id="elem">link</a>
```

Let's make it so that clicking on the link does not lead to another page:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function(event) {
4   event.preventDefault();
5   console.log('You cannot follow this link!');
6 });
```

№1

⊗jsPmOECD A

Links are given. Make it so that when you click on the link, its href is written to the its end, and there is no transition by link.

№2

⊗jsPmOECD A

Given two inputs, a paragraph and a link. Let numbers be entered into the inputs. Make it so that when you click on the link, the sum of the entered numbers is written to the paragraph.

Event bubbling in JavaScript

Imagine that you have several nested blocks:

```
1 <div id="elem1">
2   <div id="elem2">
3     <div id="elem3"></div>
4   </div>
5 </div>
```

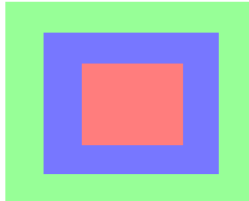
Get references to our divs into variables:

```
1 let elem1 = document.querySelector('#elem1');
2 let elem2 = document.querySelector('#elem2');
3 let elem3 = document.querySelector('#elem3');
```

And now we will hang click handlers on them:

```
1 elem1.addEventListener('click', function() {
2   alert('green');
3 });
4 elem2.addEventListener('click', function() {
5   alert('blue');
6 });
7 elem3.addEventListener('click', function() {
8   alert('red');
9 });
```

Let's display our blocks on the screen by adding some CSS to them:



Click now on the innermost red block and you will see how the event fires first in the red block, then in the blue one, then in the green one. And this is logical, because by clicking on the inner block, you simultaneously click on all the outer ones.

That is, it turns out that when you click on the innermost block, the click event fires first in it, then fires in its parent, in the parent of its parent, and so on, until it reaches the very top.

This behavior is called *bubbling* by analogy with the rising of an air bubble from the bottom. Just like the bubble, our event seems to float up, each time triggering on higher blocks.

№ 1

©jsPmOEEB

By your own, write the code that will demonstrate the bubbling of events. Check it for bubbling different types of events.

№ 2

©jsPmOEEB

Not all events may bubble up. Experimentally find at least one event that will not bubble up.

Getting the target element when an event is bubbling in JavaScript

Let's say we have two elements: a `div` and a paragraph `p` lying inside this `div`:

```
1 <div>
2   <p></p>
3 </div>
```

Get a reference to our `div` into a variable:

```
1 let div = document.querySelector('div');
```

Let's add some styling to our tags:

```
1 div {
2   padding: 20px;
3   border: 1px solid red;
4 }
5
6 p {
7   width: 200px;
8   height: 200px;
9   border: 1px solid green;
10 }
```

Now let the click handler be hung on the `div`:

```
1 div.addEventListener('click', function() {
2   console.log('click');
3 });
```

Due to the fact that our `div` has `padding`, when we click on the `div`, we can hit the paragraph, and we can get to a place where this paragraph does not exist, that is, directly on the `div`.

At the same time, in the click handler, `this` will always point to the element on which the handler is hung. In our case this is our `div`:

```
1 div.addEventListener('click', function() {
2   console.log(this); // div
3 });
```

However, we can get exactly the tag in which the event happened. To do this, we can see what is in `event.target`:

```
1 div.addEventListener('click', function(event) {
2   console.log(event.target); // or div, or paragraph
3 });
```

You can distinguish between these two options using the conditions:

```
1 let div = document.querySelector('div');
2
3 div.addEventListener('click', function(event) {
4   if (event.target.tagName === 'DIV') {
5     console.log('clicks exactly on the div');
6   }
7   if (event.target.tagName === 'P') {
8     console.log('clicks exactly on the paragraph');
9   }
10 });
```

You can use `matches` instead of `tagName`:

```
1 let div = document.querySelector('div');
2
3 div.addEventListener('click', function(event) {
4   if (event.target.matches('div')) {
5     console.log('clicks exactly on the div');
6   }
7   if (event.target.matches('p')) {
8     console.log('clicks exactly on the paragraph');
9   }
10 });
```

№ 1

Given the following elements:

```
1 <div>
2   <ul>
3     <li>1</li>
4     <li>2</li>
5     <li>3</li>
6   </ul>
7 </div>
```

```
1 div, ul, li {
2   padding: 20px;
3 }
4
5 div {
6   border: 1px solid red;
7 }
8 ul {
9   border: 1px solid orange;
10 }
11 li {
12   border: 1px solid green;
13 }
```

Attach a click handler to the div. In this handler, determine in which of the tags the event fired.

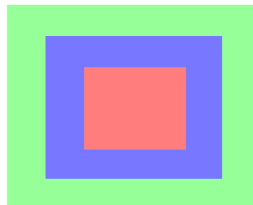
Stopping event bubbling in JavaScript

Event bubbling can be stopped on any element through which the event is bubbling. To do this, call the `stopPropagation` method of the `Event` object in the element code.

In the following example, clicking on the red block will work on itself, then on the blue block and that's it - the blue block stops further bubbling and the green block will not react in any way:

```
1 elem1.addEventListener('click', function() {
2   console.log('green');
3 });
4 elem2.addEventListener('click', function(event) {
5   console.log('blue');
6   event.stopPropagation(); // stops the propagation
7 });
8 elem3.addEventListener('click', function() {
9   console.log('red');
10 });
```

You can check:

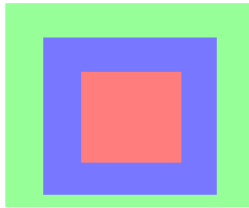


Multiple handlers on an element in JavaScript

If an element has several handlers for one event, then even if the bubbling stops, all of them will be executed. That is, `stopPropagation` prevents the event from progressing further, but all handlers will work on the current element. See example:

```
1 elem1.addEventListener('click', function() {
2   console.log('green');
3 });
4 elem2.addEventListener('click', function(event) {
5   console.log('blue - the first handler');
6   event.stopPropagation(); // stops the bubbling
7 });
8 elem2.addEventListener('click', function() {
9   console.log('blue - the second handler'); // still works
10 });
11 elem3.addEventListener('click', function() {
12   console.log('red');
13 });
```

You can check:

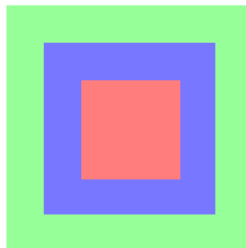


Immediate termination of bubbling in JavaScript

To completely stop processing, modern browsers support the `stopImmediatePropagation` method. It not only prevents bubbling, but also stops event processing on the current element. Let's apply it:

```
1 elem1.addEventListener('click', function() {
2   console.log('green');
3 });
4 elem2.addEventListener('click', function(event) {
5   console.log('blue - the first handler');
6   event.stopImmediatePropagation(); // stops the bubbling
7 });
8 elem2.addEventListener('click', function() {
9   console.log('blue - the second handler'); // won't work anymore
10 });
11 elem3.addEventListener('click', function() {
12   console.log('red');
13 });
```

You can check:



Usage of stopping event bubbling in JavaScript

Let inside one parent we have a button and some block:

```
1 <div id="parent">
2   <button>click me</button>
3   <div id="block">
4     text
5   </div>
6 </div>
```

Get references to our elements into variables:

```
1 let parent = document.querySelector('#parent');
2 let button = document.querySelector('button');
3 let block = document.querySelector('#block');
```

Let our block be initially hidden:

```
1 #block:not(.active) {
2   display: none;
3 }
```

Let's make it so that by clicking on the button our block appears:

```
1 button.addEventListener('click', function() {
2   block.classList.add('active');
3 });
```

And now we will make it so that by clicking on any place of the parent, our block is hidden:

```
1 parent.addEventListener('click', function() {
2   block.classList.remove('active');
3 });
```

However, we are in for an unexpected surprise: since the button is inside the parent, then *clicking on the button simultaneously means also clicking on the parent*. This means that at first our block will appear, and then, due to the bubbling of the event, the handler in the parent will work and our block will be hidden.

This is where the ability to cancel the bubbling comes in handy: we can make it so that when you click on the button to cancel the bubbling, the parent does not react to this click.

№1

©jsPmOEbUs

Implement the correct performance of the described task on your own.

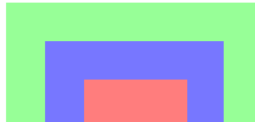
Events capturing in JavaScript

In addition to the bubbling of events, there is also *capturing*. In fact, the event first goes from top to bottom (capturing stage), reaches our element (target stage) and only then starts to bubble up (bubbling stage).

In order to hang an event handler taking into account the stage of capturing, there is a third optional parameter in `addEventListener`. If it is equal to `true` - the event will fire at the capturing stage, and if `false` - at the bubbling stage (this is by default). Let's look at an example:

```
1 elem1.addEventListener('click', function() {
2   console.log('green - capturing');
3 }, true);
4 elem1.addEventListener('click', function() {
5   console.log('green - bubbling');
6 }, false);
7
8 elem2.addEventListener('click', function() {
9   console.log('blue - capturing');
10 }, true);
11 elem2.addEventListener('click', function() {
12   console.log('blue - bubbling');
13 }, false);
14
15 elem3.addEventListener('click', function() {
16   console.log('red - capturing');
17 }, true);
18 elem3.addEventListener('click', function() {
19   console.log('red - bubbling');
20 }, false);
```

You can check:



The task is generally solved and we have avoided duplicating the code of the handler function. However, we still have to hang event handlers in two places: both in the loop for the existing `li`, and when the button is clicked. In the next lesson, we will look at a way to get rid of this inconvenience.

Attaching handlers to new elements in JavaScript

Suppose we have the list `ul` and a button:

```
1 <ul>
2   <li>item</li>
3   <li>item</li>
4   <li>item</li>
5   <li>item</li>
6   <li>item</li>
7 </ul>
8
9 <button>add</button>
```

Let's get our elements into the corresponding variables:

```
1 let button = document.querySelector('button');
2 let list   = document.querySelector('ul');
3 let items  = list.querySelectorAll('li');
```

Let's make it so that when you click on any `li`, an exclamation mark is added to the end of it:

```
1 for (let item of items) {
2   item.addEventListener('click', function() {
3     this.textContent = this.textContent + '!';
4   });
5 }
```

Let's now make it so that when the button is pressed, a new `li` is added to the end of the list:

```
1 button.addEventListener('click', function() {
2   let item = document.createElement('li');
3   item.textContent = 'item';
4   list.appendChild(item);
5 });
```

We will however get a problem: clicking on the newly added `li` will not add an exclamation mark to the end. The point is that we add a click handler only to those `li` that originally existed, but *not to new ones*.

Let's fix the problem by attaching a click handler to the new `li`:

```
1 button.addEventListener('click', function() {
2   let item = document.createElement('li');
3   item.textContent = 'item';
4
5   item.addEventListener('click', function() { // click handler
6     this.textContent = this.textContent + '!';
7   });
8
9   list.appendChild(item);
10 });
```

However, now the handler function code is duplicated in two places - for the original `li` and for the new ones. Let's fix this by moving it to a separate function:

```
1 function handler() {
2   this.textContent = this.textContent + '!';
3 }
```

```
1 function handler() {
2   this.textContent = this.textContent + '!';
3 }
```

Use our function to avoid code duplication:

```
1  for (let item of items) {
2    item.addEventListener('click', handler);
3  }
4
5  button.addEventListener('click', function() {
6    let item = document.createElement('li');
7    item.textContent = 'item';
8
9    item.addEventListener('click', handler);
10
11    list.appendChild(item);
12  });
```

The task is generally solved and we have avoided duplicating the code of the handler function. However, we still have to hang event handlers in two places: both in the loop for the existing `li`, and when the button is clicked. In the next lesson, we will look at a way to get rid of this inconvenience.

Event delegation in JavaScript

In the previous lesson, the problem that occurs when adding new elements was described and its solution was given. In this lesson, we will look at a better way to get around the problem - *event delegation*. Let's analyze it.

As you already know, by clicking on `li`, we are simultaneously clicking on `ul`. This is possible due to event bubbling. We can use this to solve our problem: we will hang an event not on each `li`, but on their parent `ul`:

```
1  list.addEventListener('click', function() {
2
3  });
```

Now in the event handler, `this` will point to the element to which the handler is attached, and `event.target` will point to the element in which the event happened:

```
1  list.addEventListener('click', function(event) {
2    console.log(this);           // our list
3    console.log(event.target);  // our list item
4  });
```

Let's make it so that an exclamation mark is added to the end of the `li`, on which the click happened:

```
1  list.addEventListener('click', function(event) {
2    event.target.textContent = event.target.textContent + '!';
3  });
```

No 1

⊗jsPmOEED

Repeat the above solution. Make sure the new `li` will also respond to the click.

Universal event delegation in JavaScript

The code in the previous lesson is working, however, not without flaws. Let's analyze these disadvantages and find a more universal solution.

The disadvantage of our code will manifest itself when there are some nested tags inside `li`. In our case, let these be the `i` tags:

```
1 <ul>
2   <li>item <i>italic</i> item</li>
3   <li>item <i>italic</i> item</li>
4   <li>item <i>italic</i> item</li>
5   <li>item <i>italic</i> item</li>
6   <li>item <i>italic</i> item</li>
7 </ul>
```

In this case, clicking on the `i` tag will add an exclamation mark to the *end of the tag i*, and not the `li` tag, as we would like - after all, exactly the tag in which the event occurred gets into `event.target`.

You can solve the problem using the method `closest`:

```
1 list.addEventListener('click', function(event) {
2   let li = event.target.closest('li');
3
4   if (li) {
5     li.innerHTML = li.innerHTML + '!';
6   }
7 });
```

№1

⊗jsPmOEVED

Repeat the above solution.